

ELI — Commands and Installers

ELI v2.5.13

October 2026

Contents

ELI v2.0 — Commands & Installers Reference **1**
1. The one-click installer (recommended for everyone) . . . 1
2. Full install from source (developers) . . . 2
3. Launching ELI . . . 3
4. Models . . . 3
5. Alternative & advanced install paths . . . 4
6. Building release packages (maintainer) . . . 4
7. Health checks & maintenance . . . 5
8. Uninstall . . . 5
CLI-reachable surfaces . . . 5
The Python environment (`scripts/eli_env.py`) . . . 6

ELI v2.0 — Commands & Installers Reference

> **Updated for v2.4.84 (September 2026).** One hardware-aware GUI wizard. Prebuilt installers on GitHub Releases. Full beginner map: `first time Installation/INSTALLATION_GUIDE.md`.

Every install path and command in one place. Copy-paste ready. Run everything from inside the ELI folder unless noted.

Canonical version: **2.4.84**. Best-tested platform: **Linux x86_64 + NVIDIA**. Windows, macOS, AMD, Intel Arc, and Apple Silicon ship supported paths; Iris Xe / ≤8 GB use CPU policy.

---

1. The one-click installer (recommended for everyone)

From a downloaded portable release

```
tar -xzf ELI_v2-2.4.84-linux-portable.tar.gz
cd ELI_v2-2.4.84-linux-portable
chmod +x ELI_Setup.sh
./ELI_Setup.sh
```

`ELI_Setup.sh` → `scripts/eli_setup.sh`:

1. Minimal `.venv` + PySide6 (when a display is available)

2. **`eli.setup.hardware_policy`** decides CPU vs GPU
3. **Unified Install Wizard** streams `install.sh` / `install.ps1` **once**
4. Hard-fail stages until nomic + voice + chat model succeed

Terminal fallback (headless / no Qt): `install.sh --yes [--cpu-only] --auto-model`, then `--run-remaining`.

From a git clone

```
git clone https://github.com/ShadowESC95/ELI_v2.0.git
cd ELI_v2.0
./scripts/eli_setup.sh
```

Linux AppImage

```
chmod +x ELI_v2-2.4.84-x86_64.AppImage
./ELI_v2-2.4.84-x86_64.AppImage
```

Compatibility aliases (not separate installers)

`scripts/eli_one_click_setup.sh` and `scripts/install_eli.sh` **only redirect** to `eli_setup.sh`.

---

2. Full install from source (developers)

Linux / macOS

```
git clone https://github.com/ShadowESC95/ELI_v2.0.git
cd ELI_v2.0
bash install.sh                 # interactive: system report → plan → install → pick model
./scripts/eli_launch.sh         # launch the desktop app
```

On Iris Xe / ≤8 GB prefer `./scripts/eli_setup.sh` or `bash install.sh --yes --cpu-only --auto-model`.

`install.sh` flags (combine as needed):

Flag	Effect
`--yes` / `-y`	No prompts — use detected defaults (CI / piped installs)
`--cpu-only`	Force the CPU build (ignore GPU)
`--gpu`	Force the GPU build even if none is auto-detected
`--install-cuda` / `--cuda`	Best-effort install the CUDA toolkit (nvcc), then rebuild llama-cpp with CUDA
`--auto-model`	Download one model sized to your VRAM after install
`--model=KEY`	Download a specific model (e.g. `--model=qwen2.5-7b`)
`--no-model`	Never download a model (also skips embedder/voice) — fully offline install
`--latest`	Use version ranges instead of the frozen reproducible lock
`--skip-torch`	Skip PyTorch (smaller install; disables self-training)

Examples:

```
bash install.sh --yes --auto-model          # hands-off, with a model
bash install.sh --cpu-only --no-model        # minimal, no GPU, add a model later
bash install.sh --install-cuda --model=phi-4 # CUDA toolkit + a specific model
```

Windows

```
install.bat            :: normal — CUDA install from the frozen lock + GPU verify
install.bat /cpu       :: CPU-only
install.bat /cuda      :: also auto-install the CUDA toolkit (winget) if missing
install.bat /latest    :: version ranges instead of the frozen lock
```

Or call the PowerShell installer directly:

```
powershell -ExecutionPolicy Bypass -File install.ps1 [-CpuOnly] [-Gpu] `
  [-InstallCuda] [-Yes] [-AutoModel] [-NoModel] [-Model qwen2.5-7b] [-Latest]
```

Launch with `eli.bat`.

Developer editable install (pip)

```
python3 -m venv .venv && . .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -e ".[full]"
python -m eli                       # GUI
python -m eli --headless            # terminal REPL
```

Android / Termux (headless — server + CLI, no GUI)

```
bash scripts/install_android.sh
```

---

3. Launching ELI

```
./scripts/eli_launch.sh                # desktop app (GUI) — default
./scripts/eli_launch.sh gui            # same, explicit
./scripts/eli_launch.sh serve          # API + web-app server (this machine only)
./scripts/eli_launch.sh serve --lan    # server exposed to your home network
./scripts/eli_launch.sh both --lan     # server in background + desktop app
./eli.sh                               # shortcut for the desktop app
.venv/bin/python -m eli                # desktop app via module
.venv/bin/python -m eli --headless     # text-only terminal REPL
```

Headless slash commands: `/status` · `/mode` · `/reset` · `/help` · `/quit`

The web / phone server directly:

```
./scripts/eli_serve.sh                  # 127.0.0.1 only  → http://127.0.0.1:8081/
./scripts/eli_serve.sh --lan            # home network → prints a token-protected URL + QR
./scripts/eli_serve.sh --port 9000      # custom port
./scripts/eli_serve.sh --lan --token MYSECRET   # use a specific access token
```

---

4. Models

```
.venv/bin/python -m eli.core.model_download --list     # show the catalog
.venv/bin/python -m eli.core.model_download --auto     # one best-fit for your VRAM
.venv/bin/python -m eli.core.model_download --choose   # multi-select menu (pick any number)
.venv/bin/python -m eli.core.model_download --aux      # just the memory embedder (~85 MB)
```

Bring your own: drop any chat/instruct `.gguf` into `models/` — ELI finds it and fits it to your hardware. Vision models also need their matching `mmproj` GGUF alongside.

Voice weights (if not fetched during install):

```
.venv/bin/python -m eli.runtime.voice_assets
```

Restore the model/voice pack from GitHub (tag `local-assets-v2.1`, needs `gh`):

```
gh auth login
./RUN_ELI.sh --with-github-assets
# or, without gh, after downloading assets manually:
python3 scripts/restore_github_asset_files.py --from-dir /path/to/downloaded/assets
```

---

5. Alternative & advanced install paths

Path	Command	When to use
One-click run (portable)	`./RUN_ELI.sh`	Daily launch of a portable install
Classic portable install	`./INSTALL_ELI.sh` then `./RUN_ELI.sh`	Portable package, manual two-step
Safe Linux install	`bash scripts/safe_install_linux.sh`	Conservative install with extra checks
Add the `eli` terminal command	`bash scripts/install_eli_command.sh`	Get `eli` on your `$PATH` (`~/.local/bin/eli`)
Install app-menu icons only	`bash scripts/install_desktop_apps.sh`	Re-add ELI / ELI Server / ELI Setup launchers
Repo venv runner	`bash scripts/run_eli_repo_venv.sh`	Run against the repo's own `.venv`
Purge a legacy install	`bash scripts/purge_legacy_eli.sh`	Clean up an older ELI before reinstalling

If your shell cached an old `eli` command after reinstalling: `hash -r`.

---

6. Building release packages (maintainer)

```
bash scripts/build_v2_release.sh                # portable Linux tar.gz
bash scripts/build_v2_release.sh --with-assets  # + bundled models (very large)
bash scripts/build_grandma_release.sh           # portable tar.gz + AppImage + checksums
bash packaging/linux/build-appimage.sh          # AppImage only (from the portable build)
bash scripts/package_eli_release.sh             # wheel + sdist
bash build_packages.sh wheel appimage windows-lean   # pick targets
```

Windows `Setup.exe` (run on a Windows PC with Inno Setup 6):

```
bash build_packages.sh windows-lean
powershell -ExecutionPolicy Bypass -File packaging/windows/build-windows.ps1 -Version 2.4.84
```

A signed/notarized macOS `.dmg` must be built on a Mac. Large model/voice binaries ship separately as GitHub Release assets (over Git's 100 MB blob limit). Full publish steps: RELEASE.md.

Installer branding

The setup wizard carries the ELI icon on every page. Three separate slots feed from the one source icon, and they are not interchangeable:

Slot	File	Where it shows
`SetupIconFile`	`packaging/desktop/Eli_Icon.ico`	the `Setup.exe` file icon itself
`WizardImageFile`	`packaging/desktop/wizard_large*.bmp`	the welcome and finish pages
`WizardSmallImageFile`	`packaging/desktop/wizard_small*.bmp`	the header of every other page

Inno Setup accepts **only 24-bit BMP** for the two wizard slots — it rejects `.png` and `.ico`, and mis-renders a BMP that carries an alpha channel — so they cannot simply be the app icon and are generated from it instead:

```
python3 packaging/desktop/generate_wizard_images.py   # regenerate after changing the icon
```

The second file in each `installer.iss` list is the high-DPI variant; Inno picks by the user's display scaling. Desktop and Start-menu shortcuts take their icon from the executable, which PyInstaller embeds via `ELI.spec`, so they need no separate entry.

The blueprint PDFs in this directory carry the same mark on their title page, applied by `scripts/generate_blueprint_pdf`

---

7. Health checks & maintenance

```
bash run_tests.sh                       # run the test suite
bash eli_diagnose.sh                    # system diagnosis report
.venv/bin/python -m eli.tools.mic_diag  # microphone diagnostics (voice input)
.venv/bin/python -m eli.core.init_data  # (re)build the local databases
```

Reading the licence

ELI is **source-available**, not open-source: PolyForm Noncommercial License 1.0.0. Run, modify, and distribute it for any noncommercial purpose; you may not use it, or a modified version, for a commercial purpose. The terms are reachable identically from every download — installed app, portable folder, AppImage or `.app`:

```
eli --license          # source install / portable tarball
./ELI_v2-*.AppImage --license
ELI.exe --license      # Windows (the Setup.exe also shows it during install)
```

The files ship alongside the app too: `LICENSE`, `NOTICE`, `THIRD_PARTY_NOTICES.md` (dependency licences, incl. the PySide6 LGPL note) and `models/MODEL_LICENSES.md` (model and voice terms).

---

8. Uninstall

ELI lives entirely inside its own folder — nothing spreads across your system:

```
rm -rf /path/to/ELI_v2.0
```

For an AppImage install, also remove `~/.local/share/ELI_v2`. If you added app-menu icons, delete the `.desktop` files from `~/.local/share/applications/`.

ELI v2.0 — © 2026 Jason Fitzgibbon Bridgeman. Source-available under the PolyForm Noncommercial License 1.0.0. Questions: jaybridgeman0095@gmail.com

CLI-reachable surfaces

Module	`python -m` entry	What it does
`eli.learning.target_registry`	—	declare/list/delete LoRA training targets
`eli.learning.review_queue`	—	mine + review training candidates
`eli.plugins.security_scan`	—	`scan_file(path)` — 11-engine malware scan
`eli.plugins.mcp`	—	`doctor()` — diagnose every configured MCP server
`eli.cognition.agent_trust`	—	`grant` / `revoke` / `inspect` custom agent code
`tools/eval/run_eval.py`	`--target router|engine|all`	eval board; now delegates to `run_board()`

`tools/eval/run_eval.py` output is unchanged — `main()` now calls `run_board()` and does the printing, so the CLI behaves exactly as before while the GUI can drive the same code in-process.

Requirements

`requirements.txt` gained `peft`, `datasets` and (non-macOS) `bitsandbytes`. See `installation.md`.

The Python environment (`scripts/eli_env.py`)

One standard-library file decides everything about the interpreter ELI runs on, for Linux, macOS and Windows. It is run by whatever Python can be found, because it is needed exactly when ELI's own environment cannot start.

- `pick`: the interpreter a new install is built with. Every Python 3 on the machine at or above the floor in `pyproject.toml` is found, and for each one PyPI is asked which locked packages have no ready-made build for it on this system (kept for a month; a release's own wheelhouse counts too). The one with the fewest wins, the newest on a tie, and the installer prints what it found per version. Offline with no wheelhouse, the newest. `install.sh` and `install.ps1` call it unless `PYTHON` is set.
- `torch-index cuda|rocm|xpu|cpu`: the PyTorch indexes with that kind of build for the Python running it, newest first, read from PyTorch's own list of builds. CUDA builds newer than the NVIDIA driver supports are left out (the driver is read by `eli_gpu_pack.py`, the same reader the frozen app uses). The installers take the first, check it runs on the GPU, and fall back to the oldest on offer, then CPU. They used to fix cu121 and rocm6.2, which stop at Python 3.12: a 3.13 or 3.14 install with an NVIDIA card got CPU PyTorch, and on Linux the inference engine was then built without CUDA too.
- `versions`: the Python versions `pyproject.toml` lists, oldest first. The Windows installer reads its floor from it and the release packages bundle wheels for each one.
- `status`: whether `.venv` can run ELI, and in plain words why not. A system upgrade that replaces the Python an environment was built with (Ubuntu 24.04 to 26.04: 3.12 to 3.14) leaves it starting as the new version and finding no packages.
- `repair`: if an interpreter of the environment's own version is still installed anywhere (PATH, the `py` launcher, uv, pyenv, Homebrew, python.org), the environment is pointed back at it and nothing is reinstalled. Otherwise it says to run the installer.
- `create`: makes `.venv`, never on a Python older than the declared floor. Debian and Ubuntu ship Python without the part that makes environments (`python3-venv`); the installers used to stop there on Python's own error. The missing package is added when that takes no password (root, or sudo without one); otherwise the user is given the one command, and no half-made `.venv` is left behind.

- `install-each ROOT FILE`: installs a requirement file one requirement at a time and names the ones that failed. pip installs a file all or nothing, so one package that cannot be compiled on a system (PyAudio without Python's headers) used to leave the environment with none of the other hundred, the GUI toolkit among them. `install.sh` falls back to this after the pinned set and the ranged set have both failed as a whole.
- `verify`: whether ELI and every package it declares it cannot run without are installed, asked of the environment itself from a folder that is not the checkout. `install.sh` used to check with `import eli` from inside the source tree, which succeeds whether or not anything was installed, and reported "installation complete" for an environment with nothing in it.

Every launcher checks before starting (`eli_env_ready` in `scripts/eli_isolate_env.sh`; `eli.bat`; `scripts/eli_serve.ps1`), so the user sees one sentence and the fix instead of a traceback. The installers run `status` on an existing `.venv`, try `repair`, and rebuild only if that fails. The frozen release packages carry their own Python and never go through this.

The Piper speech engine is found the same way round: ELI's own copy beside the running interpreter first, then PATH, and a candidate is used only if it starts and is Piper (`tts_router._find_piper_bin`). PATH used to come first and only the name was checked, so a `piper` left by a removed Python, or the unrelated mouse-configuration tool of the same name on some Linux distributions, was run as the speech engine.